

Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Análisis y Visualización de Datos Masivos / Visual Analytics and Big Data

AURA: Arquitectura local-first para auditoría inteligente de calidad del dato con diagnóstico asistido por LLM

Campo	Detalle
Trabajo fin de estudio presentado por:	Joseph David Gari Bustos
Tipo de trabajo:	Desarrollo de Software
Director/a:	Luis Guadalupe Macias Trejo
Fecha:	27-06-2026

Resumen

Este Trabajo Fin de Máster presenta una versión ampliada de la tercera entrega de AURA, una arquitectura local-first para auditoría inteligente de calidad del dato. En esta etapa, el prototipo ya cubre un flujo operativo visible que comprende carga local del archivo, perfilamiento del dataset, diagnóstico asistido, preparación de script, revisión humana y exportación de evidencia. Además, la herramienta opera con varios proveedores de inteligencia artificial, incluyendo opciones cloud, locales y experimentales en navegador, siempre bajo un esquema de evidencia y gobernanza humana.

En esta versión se refuerza la madurez documental del proyecto. Se explican los componentes técnicos en lenguaje académico, se incorporan capturas de evidencia del prototipo, se presentan tablas de resultados sobre Titanic y sobre un dataset sintético con ground truth, y se distinguen con claridad los resultados consolidados de aquellos que permanecen como experimentales. La validación controlada del motor determinista sobre `synthetic_ground_truth.csv` evaluó 16 reglas y 16 hallazgos, con criterios mínimos de precisión y recall iguales o superiores a 0,93 y F1-score igual o superior a 0,90. En Titanic, el sistema procesó 891 filas y 12 columnas, generando 10 hallazgos deterministas y una cadena de evidencia que permite pasar del perfilamiento al diagnóstico asistido, plan de remediación y revisión humana.

El alcance de esta entrega no afirma que AURA corrija definitivamente el dataset ni que ejecute scripts de limpieza en producción. La mejora efectiva solo podrá sostenerse cuando se ejecute el script aprobado sobre una copia del dataset, se reaudite el resultado y se compare el estado antes y después. La contribución actual es demostrar una arquitectura reproducible, gobernada y explícita sobre sus límites.

Palabras clave: calidad del dato; inteligencia artificial centrada en datos; modelos de lenguaje; local-first; revisión humana.

Abstract

This Master Thesis presents an extended version of the third submission of AURA, a local-first architecture for intelligent data quality auditing. At this stage, the prototype already exposes an operational flow that covers local file loading, dataset profiling, assisted diagnosis, script preparation, human review and evidence export. The tool also supports multiple AI providers, including cloud, local and browser-based experimental options, all governed through evidence restrictions and human oversight.

This version strengthens the academic maturity of the project. Technical components are explained in natural language, visual evidence of the prototype is included, results are reported for Titanic and a synthetic ground-truth dataset, and consolidated results are distinguished from experimental ones. The controlled validation of the deterministic engine over `synthetic_ground_truth.csv` evaluated 16 rules and 16 findings, with minimum precision and recall values greater than or equal to 0.93 and an F1-score greater than or equal to 0.90. On Titanic, the system processed 891 rows and 12 columns, generating 10 deterministic findings and an evidence chain from profiling to assisted diagnosis, remediation planning, and human review.

The scope of this submission does not claim that AURA definitively corrects the dataset or executes cleaning scripts in production. Effective improvement can only be claimed after executing an approved script over a copy of the dataset, re-auditing the result, and comparing the before-and-after state. The current contribution is a reproducible, governed architecture that is explicit about its limits.

Keywords: data quality; data-centric AI; large language models; local-first; human-in-the-loop.

Índice de contenidos

1. Introducción

1.1. Motivación

1.2. Planteamiento del trabajo

1.3. Respuesta a la retroalimentación y alcance

2. Contexto y estado del arte

2.1. Contexto del problema

2.2. Estado del arte

2.3. Comparativa de herramientas frente a AURA

3. Objetivos concretos y metodología de trabajo

3.1. Objetivo general

3.2. Objetivos específicos

3.3. Metodología del trabajo

4. Marco normativo

5. Desarrollo específico de la contribución

5.1. Requisitos

5.2. Definición de componentes de AURA

5.3. Arquitectura y funcionamiento

5.4. Evidencia visual del prototipo

5.5. Fragmentos de código explicados

6. Código fuente y datos analizados

7. Resultados consolidados

8. Conclusiones

9. Limitaciones y prospectiva

Referencias bibliográficas

Anexos

Índice de figuras

- Figura 1. Arquitectura funcional local-first de AURA.
- Figura 2. Arquitectura actual basada en evidencia, diagnóstico, decisión y ejecución controlada.
- Figura 3. Ejemplo de perfilamiento local de un dataset tabular.
- Figura 4. Ejemplo de diagnóstico asistido con entrada controlada.
- Figura 5. Configuración de proveedor y contrato técnico de diagnóstico.
- Figura 6. Laboratorio experimental de comparación de modelos.
- Figura 7. Configuración local de modelos y control de envío de datos.
- Figura 8. Diagnóstico generado con proveedor local Ollama (qwen2.5:3b).
- Figura 9. Preparación de propuesta de script basada en hallazgos y diagnóstico.
- Figura 10. Revisión humana previa a la aprobación y exportación del script.

Índice de tablas

- Tabla 1. Comparativa conceptual entre herramientas de calidad del dato y AURA.
- Tabla 2. Diccionario funcional de componentes de AURA.
- Tabla 3. Resultados sobre Titanic.
- Tabla 4. Resultados sobre dataset con ground truth.
- Tabla 5. Estado del laboratorio de comparación de modelos.
- Tabla 6. Relación entre objetivos, evidencia y límites.

1. Introducción

Trabajar con datos de baja calidad es uno de los problemas más costosos y menos visibles que enfrentan las organizaciones. Aunque durante años la atención se centró en mejorar los algoritmos, la práctica ha demostrado que un modelo sofisticado sobre datos defectuosos produce resultados igualmente defectuosos. La integridad de los datos no es un detalle técnico secundario; es la condición de partida para que cualquier sistema de analítica avanzada o aprendizaje automático funcione de forma confiable.

1.1. Motivación

El impacto económico de una gestión deficiente de la información es significativo. La literatura profesional y académica vincula la mala calidad del dato con pérdidas económicas, retrasos en proyectos de analítica, fallos en sistemas de inteligencia artificial y decisiones organizacionales basadas en información incompleta o inconsistente. En este contexto, la calidad del dato deja de ser una tarea previa y aislada, y pasa a convertirse en un componente central de la ingeniería de datos y de la inteligencia artificial centrada en datos.

La situación se vuelve más crítica con el crecimiento de los modelos generativos. Un sistema de IA no corrige automáticamente datos defectuosos; en muchos casos amplifica sus sesgos, inconsistencias o ausencias. Por ello, resulta necesario construir herramientas que permitan observar el estado del dataset, explicar los hallazgos, proponer acciones y mantener supervisión humana antes de cualquier transformación.

1.2. Planteamiento del trabajo

AURA se plantea como una arquitectura local-first para auditoría inteligente de calidad del dato. Su objetivo no es sustituir al analista ni delegar la limpieza en un modelo de lenguaje. Su contribución consiste en organizar una cadena de trabajo donde cada etapa tiene una responsabilidad clara: el motor determinista detecta hechos, el paquete de evidencia limita qué información se entrega al diagnóstico asistido, el plan de remediación propone acciones bajo reglas controladas, y la revisión humana decide antes de convertir acciones en código.

La tercera entrega se enfoca en madurar la explicación académica y técnica de AURA. En lugar de presentar nombres internos sin contexto, este documento define cada componente, muestra su papel dentro del flujo general y expone qué evidencia sostiene cada afirmación. Además, se

incorporan resultados sobre dos tipos de datasets: Titanic, como caso aplicado y reproducible, y `synthetic_ground_truth.csv`, como dataset controlado con errores conocidos para medir el comportamiento del motor determinista.

1.3. Respuesta a la retroalimentación y alcance

La retroalimentación recibida señaló la necesidad de elevar el rigor académico, reforzar el estado del arte, documentar con mayor precisión la metodología experimental, vincular objetivos con evidencia y distinguir resultados formales de resultados preliminares. Esta versión responde a esas observaciones mediante tres decisiones: primero, definir los componentes de AURA en lenguaje claro; segundo, presentar resultados en tablas verificables; tercero, separar con cuidado lo que la herramienta ya demuestra de lo que todavía debe validarse en la fase final.

El alcance de esta entrega llega hasta la generación de evidencia, diagnóstico estructurado, plan de acciones, revisión humana y preparación de scripts revisables. No se afirma todavía una mejora real del dataset, porque no se ha ejecutado el script aprobado ni se ha realizado una reauditoría posterior. Esta decisión no debilita la entrega; por el contrario, evita confundir una propuesta de limpieza con una limpieza aplicada y medida.

2. Contexto y estado del arte

2.1. Contexto del problema

En el panorama tecnológico actual, el éxito de los sistemas de inteligencia artificial y analítica de datos depende de la integridad de la información que consumen. Los datasets pueden contener nulos, duplicados, formatos inválidos, columnas ambiguas, categorías raras, valores extremos o inconsistencias de negocio. Estos defectos no siempre impiden que un sistema funcione; muchas veces permiten que el flujo continúe, pero degradan silenciosamente la calidad de los reportes, modelos y decisiones.

Las herramientas actuales de calidad del dato suelen cubrir una parte del problema. Algunas son muy útiles para limpieza manual, otras para validación con reglas, y otras para observabilidad de pipelines. Sin embargo, pocas integran en un mismo flujo la auditoría reproducible, la interpretación asistida por modelos de lenguaje, la privacidad operativa y la decisión humana antes de ejecutar transformaciones.

2.2. Estado del arte

La inteligencia artificial centrada en datos plantea que la mejora de los sistemas de IA no depende únicamente de ajustar modelos, sino de mejorar sistemáticamente los datasets. Bajo esta mirada, la auditoría de calidad no es un trámite inicial, sino un ciclo continuo de observación, corrección y evaluación. AURA se alinea con este enfoque al priorizar la evidencia del dataset antes de cualquier interpretación asistida.

La norma ISO/IEC 25012 ofrece un marco conceptual para hablar de calidad del dato en términos de exactitud, completitud, consistencia, actualidad, trazabilidad y otras dimensiones. En AURA, estas dimensiones se traducen en reglas, hallazgos, severidades y reportes que permiten describir el estado inicial del dataset de forma reproducible.

Los modelos de lenguaje aportan una capacidad valiosa: pueden explicar hallazgos técnicos en lenguaje más comprensible y priorizar problemas. Sin embargo, también pueden producir conclusiones no soportadas, inventar columnas o sugerir acciones inapropiadas. Por esta razón, AURA no permite que el modelo actúe libremente. El diagnóstico asistido se alimenta de un paquete de evidencia controlado y la remediación se gobierna por reglas y revisión humana.

2.3. Comparativa de herramientas frente a AURA

Herramienta / enfoque	Fortaleza principal	Limitación observada	Diferencia de AURA
OpenRefine	Limpieza exploratoria e interacción manual	Depende de intervención manual y no está centrada en diagnóstico asistido	AURA genera hallazgos reproducibles, explicación asistida y evidencia exportable.
Great Expectations	Validación robusta mediante reglas y expectativas	Requiere configuración técnica y reglas previamente definidas	AURA combina reglas deterministas con interpretación asistida y revisión humana.
Monte Carlo / observabilidad	Monitoreo de linaje, anomalías y data downtime	Orientación cloud y menor foco en explicación local del hallazgo	AURA prioriza procesamiento local-first y evidencia comprensible para el usuario.
Soda Core	Validación en pipelines CI/CD	Menor cobertura semántica y dependencia de checks declarativos	AURA añade diagnóstico asistido, scripts revisables y control humano.
AURA	Auditoría local-first con evidencia determinista y diagnóstico asistido	Benchmark multimodelo aún experimental	Integra privacidad, trazabilidad, revisión humana y laboratorio comparativo.

Tabla 1. Comparativa conceptual entre herramientas de calidad del dato y AURA.

La brecha identificada no consiste en la ausencia de herramientas, sino en la falta de un flujo integrado que preserve la reproducibilidad de las reglas, restrinja el diagnóstico asistido, mantenga trazabilidad de referencias y exija decisión humana antes de ejecutar cambios.

3. Objetivos concretos y metodología de trabajo

3.1. Objetivo general

Consolidar AURA como una arquitectura local-first, reproducible y evaluable para auditoría inteligente de calidad del dato, capaz de generar evidencia determinista, restringir el diagnóstico asistido mediante paquetes de evidencia, producir planes de remediación gobernados, habilitar revisión humana y preparar una ejecución controlada y trazable.

3.2. Objetivos específicos

1. Consolidar una arquitectura local-first que cargue, procese y audite datasets CSV en el navegador, generando trazas y minimizando la información compartida con proveedores externos.
2. Diseñar y evaluar un motor determinista basado en reglas explícitas, heurística de tipos y estadística descriptiva, documentando verdaderos positivos, falsos positivos, falsos negativos, precisión, recall y F1.
3. Implementar una capa de diagnóstico asistido restringida a evidencia estructurada, evitando que el modelo opere sobre el dataset completo o genere transformaciones sin control.
4. Implementar un laboratorio de comparación de modelos bajo un contrato común, diferenciando claramente pruebas operativas, resultados experimentales y benchmark formal.
5. Implementar gobernanza human-in-the-loop mediante revisión y aprobación humana antes de generar o ejecutar cualquier script de limpieza.
6. Preparar scripts Python/Pandas revisables a partir de acciones aprobadas, manteniendo trazabilidad entre hallazgos, decisiones y código propuesto.

3.3. Metodología del trabajo

La metodología combina CRISP-DM, desarrollo ágil y evaluación experimental por fases. CRISP-DM permite estructurar comprensión del problema, comprensión de datos, preparación, evaluación y retroalimentación. Scrum permite avanzar incrementalmente en módulos de software verificables.

La evaluación experimental permite diferenciar entre evidencia formal, evidencia preliminar y elementos pendientes.

En esta entrega se utilizan dos clases de datasets. Titanic funciona como caso aplicado y reproducible para mostrar carga local, perfilamiento, hallazgos y flujo de revisión. `synthetic_ground_truth.csv` funciona como dataset controlado, porque contiene anomalías esperadas que permiten medir el comportamiento del motor determinista con métricas de clasificación.

Las ejecuciones se clasifican en cuatro estados: planificada, intentada con fallo, preliminar válida y formal válida. Una corrida de modelos solo puede considerarse formal si usa el mismo dataset, contrato común, temperatura controlada, repeticiones suficientes y exportación de resultados comparables. Por esa razón, el laboratorio de modelos se presenta como módulo experimental y no como benchmark definitivo.

4. Marco normativo

4.1. Protección de datos de carácter personal

El desarrollo de herramientas basadas en inteligencia artificial que procesan conjuntos de datos exige considerar la protección de datos personales, la seguridad de la información y la trazabilidad. En auditorías reales pueden aparecer nombres, identificadores, direcciones, registros administrativos, datos financieros o datos de salud. Por ello, AURA adopta un enfoque local-first: el CSV se carga y perfila en el navegador antes de cualquier interacción con proveedores externos.

4.2. Minimización y procesamiento local

El diseño local-first reduce exposición, pero no elimina automáticamente obligaciones normativas. Si el dataset contiene datos personales, siguen siendo relevantes los principios de licitud, transparencia, limitación de finalidad, minimización, integridad y confidencialidad. AURA contribuye a la minimización porque el diagnóstico asistido no recibe el dataset completo, sino un paquete reducido de evidencia con columnas, hallazgos y muestras limitadas.

4.3. Ética en IA y gobernanza

La gobernanza se expresa en la revisión humana. AURA no modifica datos de forma opaca ni permite que el modelo de lenguaje ejecute transformaciones autónomas. El sistema propone, valida y

muestra; la persona revisa y decide. Este enfoque favorece explicabilidad, trazabilidad y responsabilidad humana sobre transformaciones potencialmente sensibles.

5. Desarrollo específico de la contribución

Este capítulo describe la contribución como trabajo de desarrollo de software. La explicación se organiza de forma gradual: primero se presentan requisitos, después se definen los componentes técnicos en lenguaje académico, luego se muestra la arquitectura y finalmente se incorporan capturas y fragmentos de código explicados.

5.1. Identificación de requisitos

El problema que guía AURA es el deterioro silencioso de la calidad del dato antes de su uso en reportes, modelos o procesos de decisión. El usuario objetivo es un analista, ingeniero de datos, auditor técnico o equipo de datos que necesita comprender la salud de un CSV sin enviar el archivo completo a un servidor externo.

5.1.1. Requisitos funcionales

- Cargar archivos CSV localmente desde el navegador.
- Perfilar el dataset mediante reglas deterministas.
- Presentar hallazgos con severidad, columna afectada, conteo y evidencia.
- Construir un paquete estructurado de evidencia para el diagnóstico asistido.
- Generar un plan de acciones de remediación bajo políticas controladas.
- Permitir aprobación o rechazo humano de acciones.
- Preparar scripts Python/Pandas revisables desde acciones aprobadas.
- Documentar resultados, límites y evidencia exportable.

5.1.2. Requisitos no funcionales

- Privacidad desde el diseño: el archivo crudo se procesa localmente por defecto.
- Reproducibilidad: los hallazgos deben depender de reglas explícitas y no de interpretación probabilística.
- Trazabilidad: cada etapa debe indicar qué recibe, qué produce y qué evidencia deja.
- Gobernanza humana: ninguna acción destructiva debe avanzar sin aprobación.
- Claridad: los términos técnicos deben explicarse antes de usarse en la memoria.

5.2. Diccionario funcional de componentes de AURA

Para evitar que el documento dependa de nombres internos del repositorio, la Tabla 2 traduce cada componente técnico a un concepto académico y operativo. A partir de esta tabla, los nombres técnicos se usan únicamente como referencia complementaria.

Nombre técnico	Nombre académico	Función	Riesgo que controla
csvService.ts	Módulo de lectura local	Lee y convierte el CSV en datos utilizables dentro del navegador.	Evita depender de un servidor para iniciar el análisis.
auditEngine.ts	Motor determinista de auditoría	Aplica reglas explícitas para detectar nulos, duplicados, espacios invisibles, formatos inválidos y otros patrones.	Reduce variabilidad y permite repetir el mismo análisis sobre el mismo archivo.
executionEvidence.ts	Registro de evidencia de ejecución	Guarda metadatos del análisis: archivo, columnas, tiempos, fingerprint y trazas.	Permite explicar de dónde salió cada resultado.
AuditReport	Informe técnico de auditoría	Resume hallazgos, severidad, columnas afectadas, conteos y score de salud.	Evita diagnósticos vagos o puramente narrativos.
EvidenceEnvelopeV2	Paquete estructurado de evidencia	Empaqueta solo la evidencia necesaria para el diagnóstico asistido.	Evita entregar el dataset completo al modelo y reduce alucinaciones por falta de contexto.
DiagnosisResponseV2	Diagnóstico asistido estructurado	Organiza la interpretación del modelo de lenguaje en campos verificables.	Reduce respuestas libres, columnas inventadas y conclusiones sin soporte.
RemediationPlanV2	Plan de remediación gobernado	Convierte hallazgos en acciones posibles, clasificadas por riesgo y revisables por el usuario.	Evita que el modelo autorice transformaciones por sí solo.
ScriptContractV2	Contrato verificable de script	Representa qué acciones aprobadas se convierten en código Python/Pandas y cómo se verifica su integridad.	Evita scripts no trazables o modificados sin control.
Renderer determinista	Generador controlado de código	Produce código desde acciones aprobadas, no desde texto libre del modelo.	Reduce riesgo de código arbitrario o no autorizado.
HITL	Revisión humana antes de aprobar	Permite aprobar o rechazar acciones antes de continuar.	Mantiene supervisión humana sobre decisiones de limpieza.
BenchmarkLab	Laboratorio de comparación de modelos	Permite comparar proveedores y modelos bajo un flujo común.	Evita seleccionar modelos solo por intuición o disponibilidad.

Tabla 2. Diccionario funcional de componentes de AURA.

5.3. Arquitectura y funcionamiento general

AURA funciona como una cadena de responsabilidades. Primero, el archivo se carga localmente. Después, el motor determinista produce un informe técnico. Luego, se construye un paquete de

evidencia reducido para el diagnóstico asistido. A partir de esa interpretación, el sistema prepara un plan de remediación; la persona revisa y aprueba acciones; finalmente, las acciones aprobadas pueden convertirse en un script revisable.

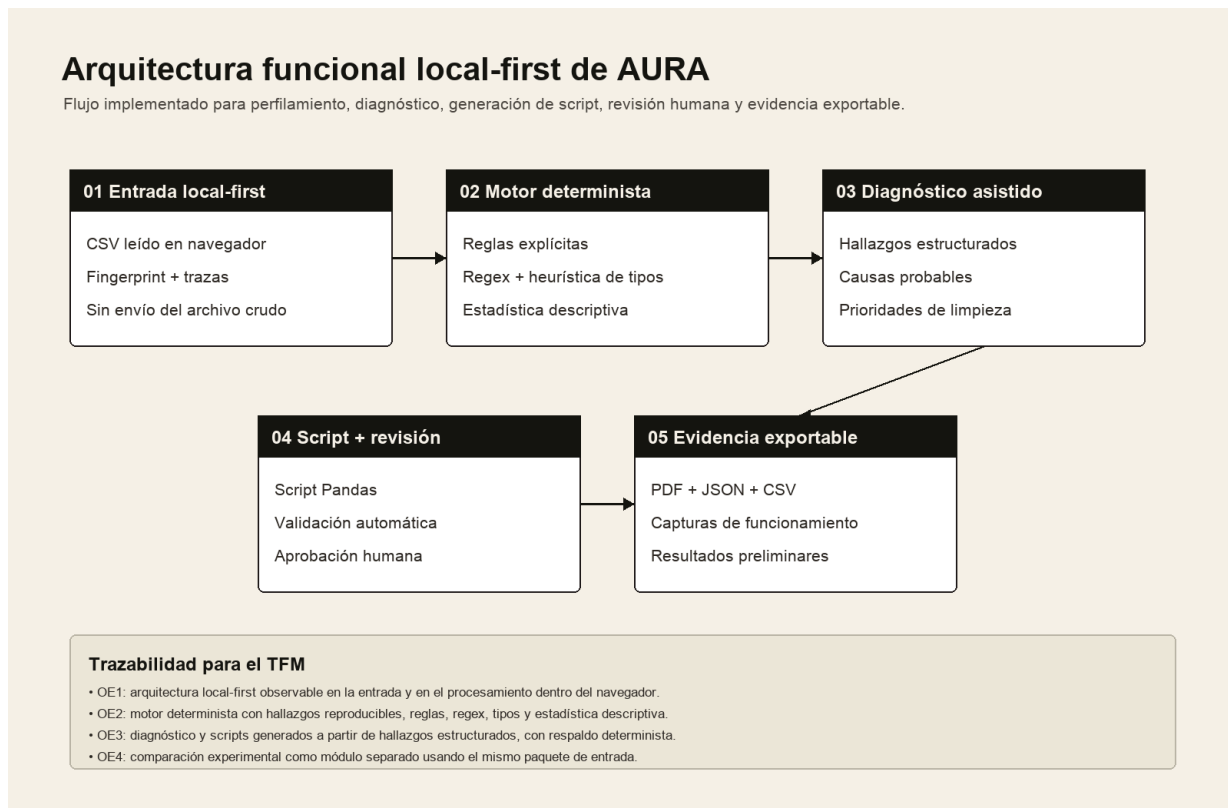
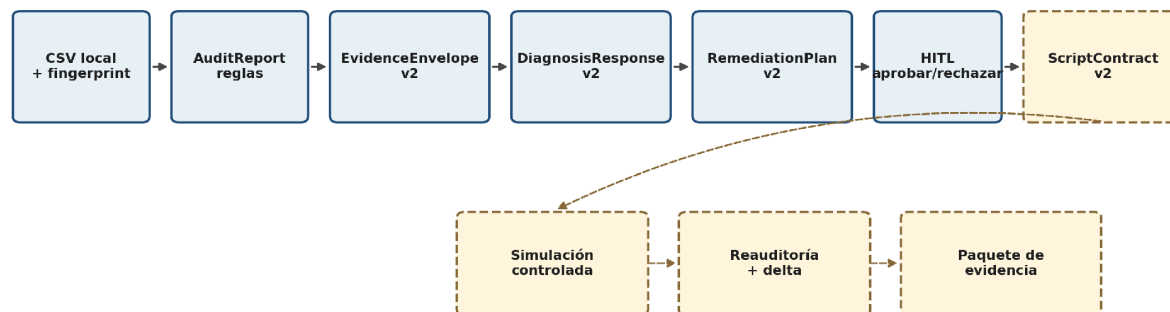


Figura 1. Arquitectura funcional local-first de AURA.

La Figura 1 resume la primera arquitectura funcional de AURA. En ella se observa una secuencia clara: entrada local, motor determinista, diagnóstico asistido, script con revisión y evidencia exportable. Esta arquitectura ya evita enviar el archivo crudo como requisito de funcionamiento y sitúa la revisión humana como parte del flujo.

Arquitectura actual de AURA basada en contratos verificables

Línea continua: implementado y auditado. Línea discontinua: siguiente cierre técnico.



Principio de seguridad: el LLM interpreta evidencia; la política determinista decide; la persona aprueba; el renderer ejecuta.

Figura 2. Arquitectura actual basada en evidencia, diagnóstico, decisión y ejecución controlada.

La Figura 2 representa la evolución conceptual de la herramienta. La diferencia más importante es que AURA ya no se describe como un flujo lineal donde el modelo produce directamente un script. Ahora se plantea como una arquitectura por etapas: evidencia, diagnóstico, remediación, revisión humana y preparación de ejecución controlada. Los bloques discontinuos representan actividades que se completarán en la fase final: ejecución segura, reauditoría y comparación antes/después.

5.3.1. Flujo operativo actual de AURA

En su estado actual, AURA presenta al usuario una secuencia de trabajo explícita: Carga, Perfil, Diagnóstico, Script, Revisión y Exportar. La etapa de Carga recibe el archivo CSV sin enviarlo por defecto a un servidor. La etapa de Perfil resume dimensiones, columnas y hallazgos preliminares del dataset. La etapa de Diagnóstico interpreta dichos hallazgos y propone hipótesis y acciones en lenguaje comprensible. Luego, la etapa de Script prepara una propuesta revisable de limpieza en Python/Pandas a partir de las acciones aprobables. La etapa de Revisión exige intervención humana antes de aprobar la propuesta, y la etapa de Exportar organiza la salida del proceso en forma de script, evidencia y artefactos de seguimiento.

5.3.2. Proveedores de diagnóstico actualmente soportados

AURA no depende de un único proveedor de inteligencia artificial. La implementación actual permite trabajar con proveedores cloud, locales y experimentales en navegador. Entre los proveedores documentados se encuentran Gemini y DeepSeek como opciones cloud, Ollama como opción local para ejecutar modelos dentro del entorno del usuario, y alternativas experimentales en navegador como Chrome Prompt API y WebLLM. Esta diversidad tiene una implicación metodológica importante: el valor de AURA no reside en casarse con un modelo concreto, sino en mantener un mismo marco de evidencia, restricciones, revisión humana y comparabilidad entre proveedores.

5.4. Evidencia visual del prototipo

Las siguientes capturas ilustran el funcionamiento visible del prototipo en su estado actual. Además de la carga y el perfilamiento, se incorporan evidencias de diagnóstico asistido con proveedor local, preparación de script y revisión humana. Las imágenes se usan como evidencia funcional del flujo observado por el usuario y se explican en lenguaje académico, diferenciando siempre entre lo que la interfaz muestra y aquello que solo podrá afirmarse después de una ejecución controlada y reauditoría.

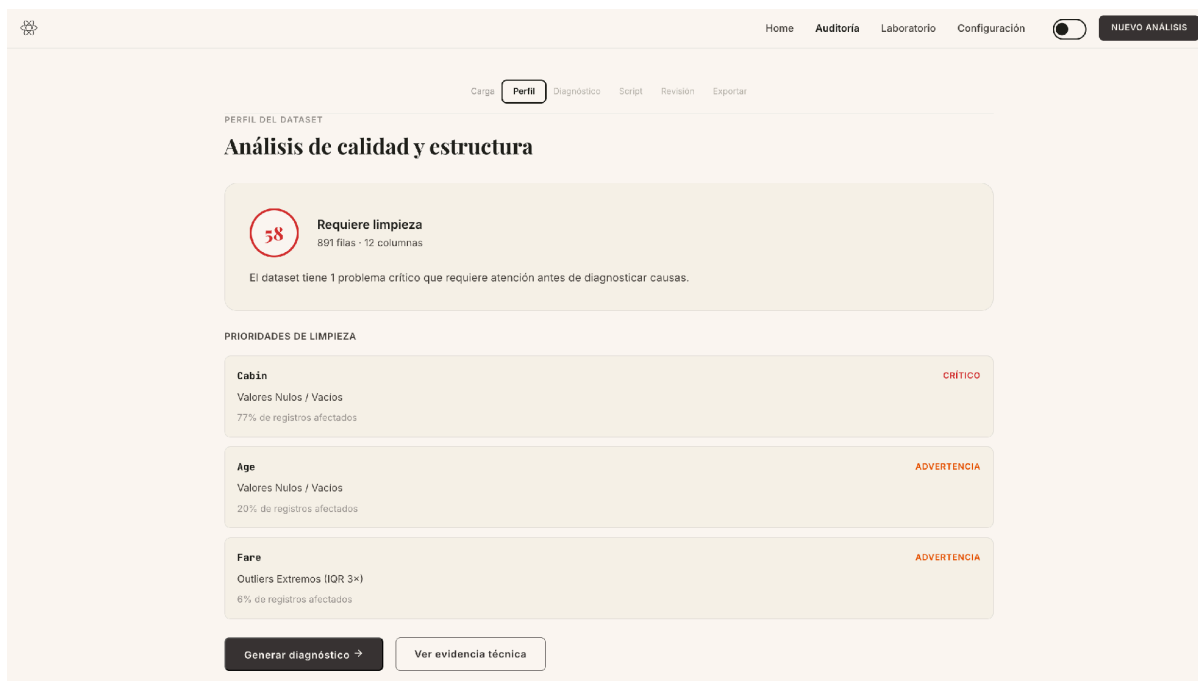


Figura 3. Ejemplo de perfilamiento local de un dataset tabular en AURA.

La Figura 3 muestra la etapa de perfilamiento. El sistema resume dimensiones, score de calidad, severidad de hallazgos, categorías afectadas, tipos y duplicados. Esta pantalla corresponde a la capa factual: todavía no hay interpretación asistida ni transformación de datos. Su valor es generar una base objetiva para las etapas posteriores.

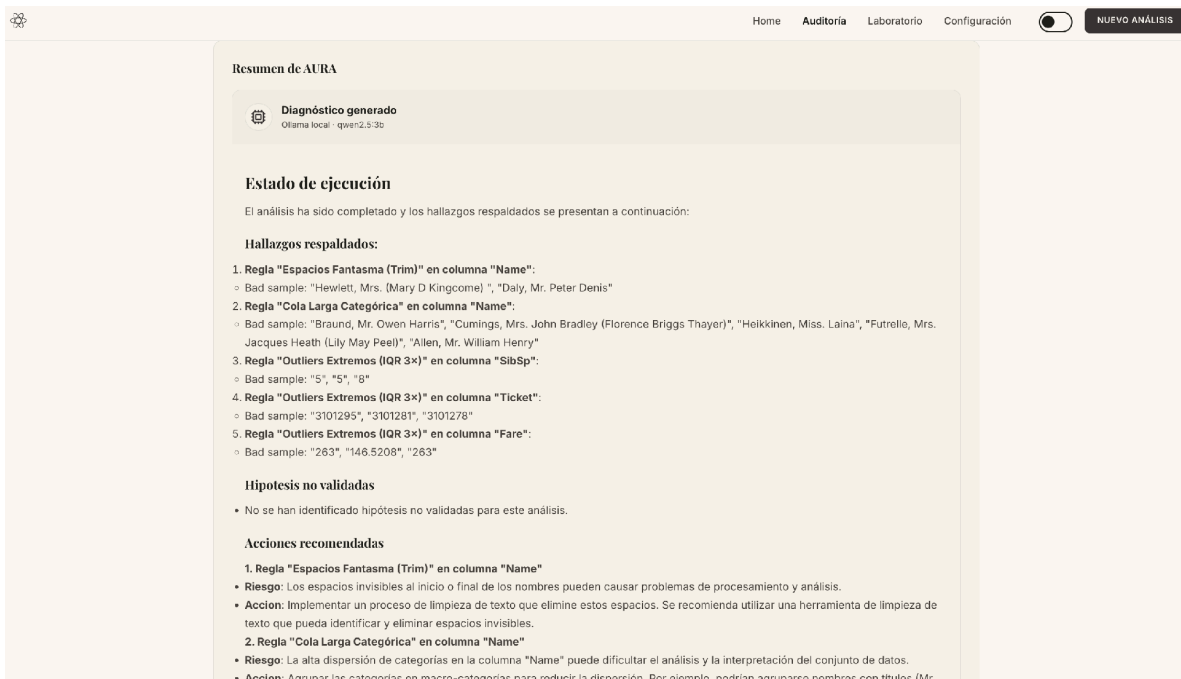


Figura 4. Ejemplo de diagnóstico asistido con entrada controlada.

La Figura 4 muestra una etapa de diagnóstico asistido. El sistema conserva a la vista el número de hallazgos recibidos, el modo de ejecución y el paquete de entrada. Esta pantalla refleja una decisión de diseño importante: el modelo no recibe libertad total sobre el dataset, sino un resumen estructurado de problemas observados por el motor determinista.

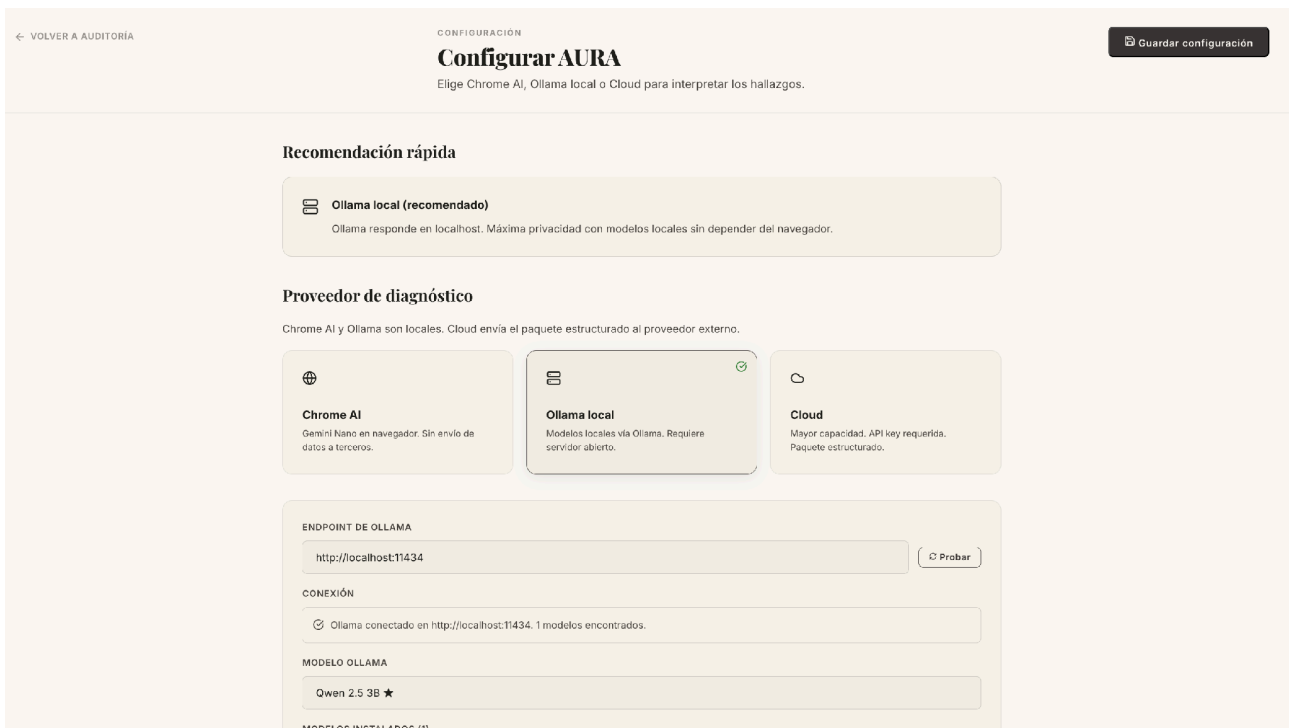


Figura 5. Configuración de proveedor y contrato técnico de diagnóstico.

La Figura 5 evidencia que AURA permite seleccionar proveedor local, cloud o experimental, pero mantiene controles sobre temperatura, modo de ejecución y contrato técnico. Esta configuración sirve para que las pruebas sean más comparables y para informar al usuario sobre el modo de procesamiento elegido.

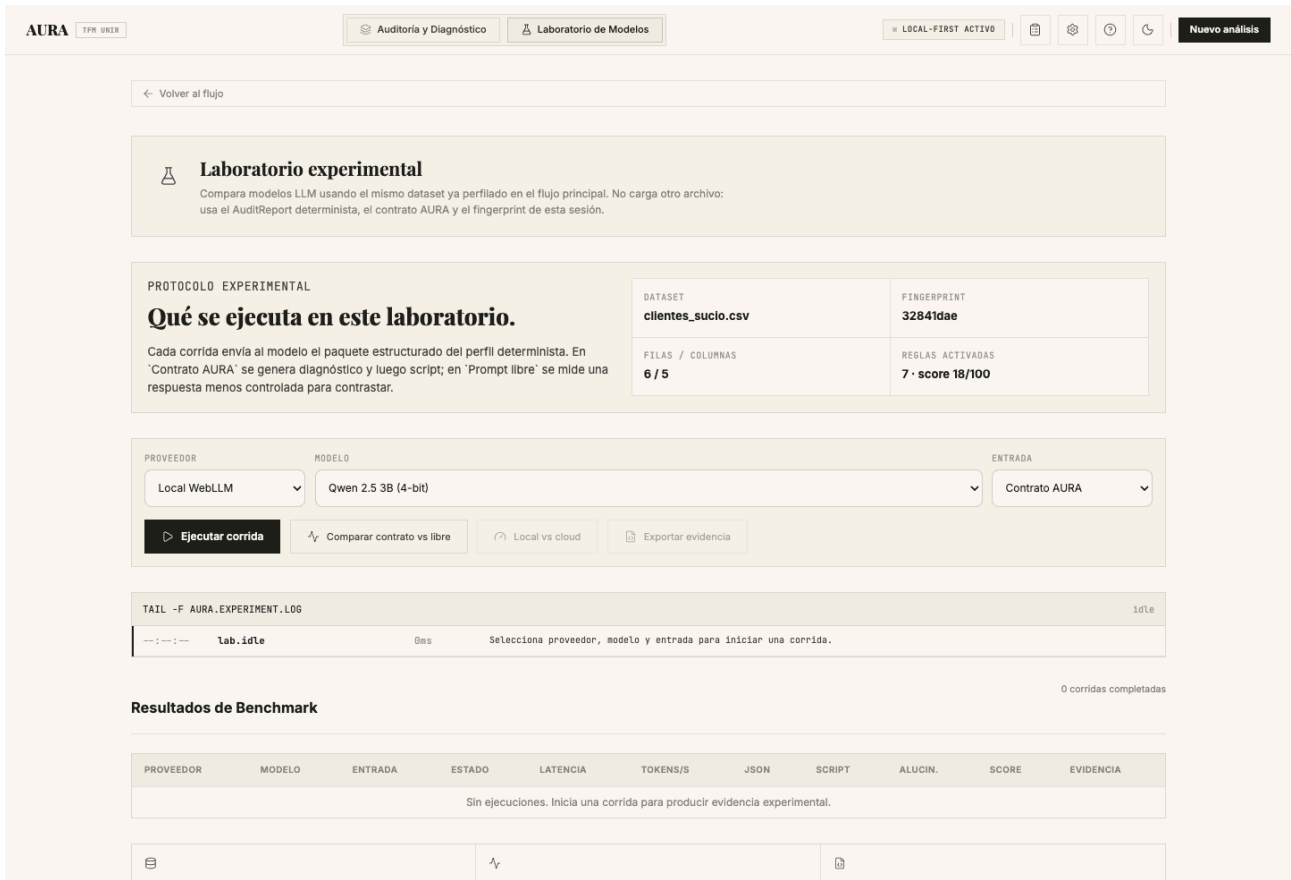


Figura 6. Laboratorio experimental de comparación de modelos.

La Figura 6 muestra el laboratorio experimental de modelos. Este módulo permite comparar proveedores bajo un mismo paquete de entrada. En esta entrega se documenta como infraestructura experimental, no como benchmark formal. No se declara un modelo ganador porque faltan repeticiones suficientes, ground truth común y exportaciones homogéneas.

CONFIGURACIÓN

Configurar AURA

Elige Chrome AI, Ollama local o Cloud para interpretar los hallazgos.



Chrome AI
Gemini Nano en navegador. Sin envío de datos a terceros.



Ollama local
Modelos locales vía Ollama. Requiere servidor abierto.



Cloud
Mayor capacidad. API key requerida. Paquete estructurado.

ENDPOINT DE OLLAMA

[Probar](#)

CONEXIÓN

☒ Ollama conectado en http://localhost:11434. 1 modelos encontrados.

MODELO OLLAMA

MODELOS INSTALADOS (1)

qwen2.5:3b
1.9 GB

DESCARGAR MODELO

[Descargar](#)

Sugeridos: qwen2.5:3b, llama3.2:3b, mistral:7b, gemma2:2b, phi3:mini

 Ollama ejecuta modelos locales en tu máquina. Requiere tener Ollama instalado y abierto. Modelos sugeridos: qwen2.5:3b, llama3.2:3b. No se descarga ningún modelo automáticamente.

Figura 7. Configuración local de modelos y control de envío de datos.

La Figura 7 muestra la configuración local del modelo. Esta pantalla es relevante para el marco normativo porque permite distinguir entre ejecución local y uso de proveedores externos. AURA reduce exposición mediante procesamiento local, pero cuando se activa un proveedor cloud debe explicarse que puede salir del dispositivo un paquete de evidencia limitado.

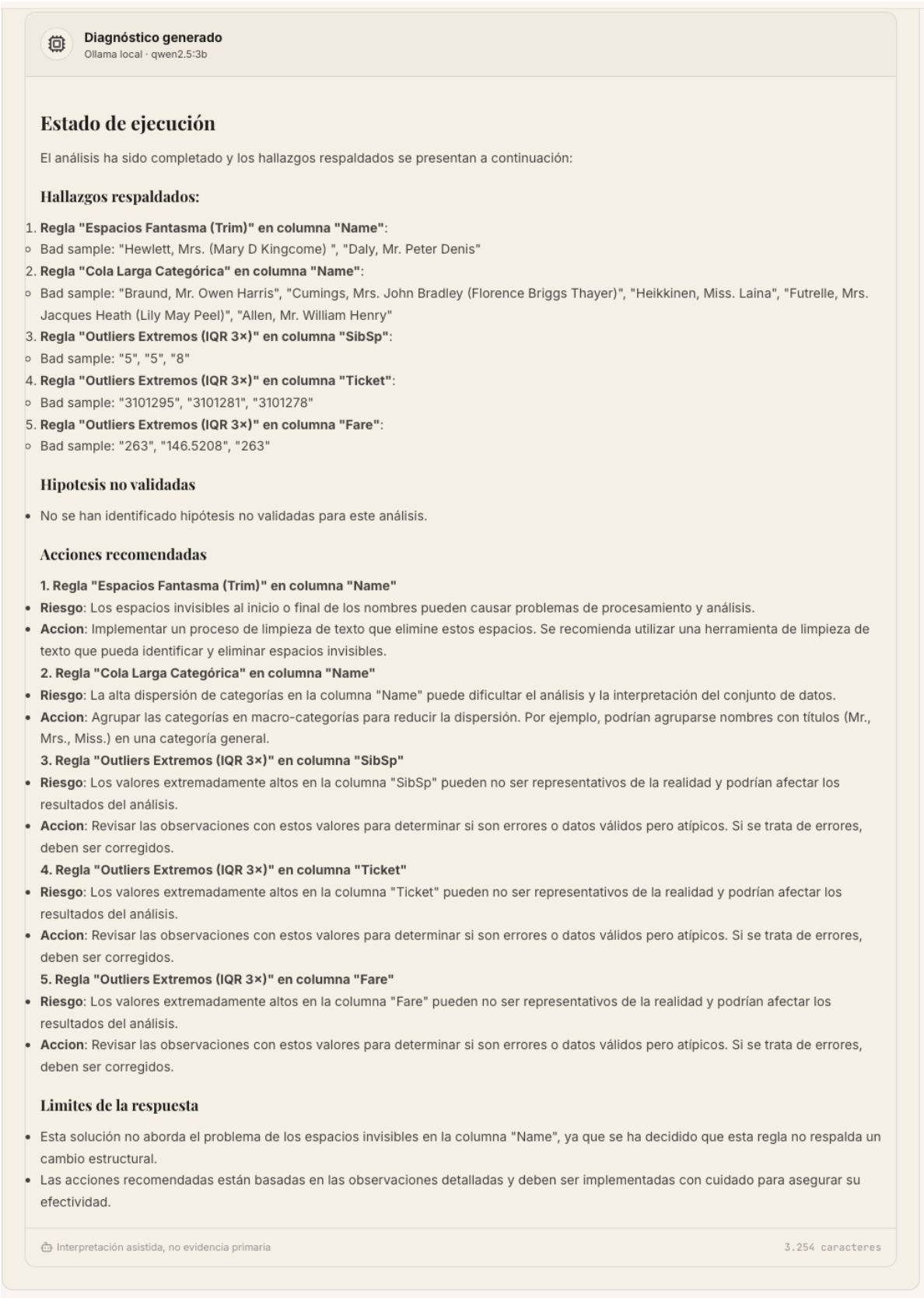


Figura 8. Diagnóstico generado con proveedor local Ollama (qwen2.5:3b).

La Figura 8 muestra una ejecución de la etapa de diagnóstico usando un proveedor local de inteligencia artificial: Ollama con el modelo qwen2.5:3b. En la pantalla se observan hallazgos respaldados, hipótesis no validadas, acciones recomendadas y límites de la respuesta. Esta evidencia es relevante porque demuestra que AURA ya no se limita al perfilamiento, sino que puede transformar hallazgos deterministas en una interpretación asistida estructurada, manteniendo separados los hechos observados, los riesgos asociados y las recomendaciones sugeridas por el modelo.



Figura 9. Etapa de preparación de propuesta de script en AURA.

La Figura 9 corresponde a la etapa Script. En ella AURA prepara una propuesta de limpieza revisable en Python/Pandas a partir de los hallazgos base y del diagnóstico disponible. La interfaz informa el estado de seguridad, la cobertura estimada, las alertas pendientes y una vista previa del código generado. Esta pantalla es importante porque deja claro que el script se presenta como una ayuda revisable, no como una corrección automática aplicada sobre el dataset original.



Figura 10. Revisión humana previa a la aprobación y exportación del script.

La Figura 10 muestra la etapa de Revisión. Aquí el usuario inspecciona la propuesta, verifica el estado del script y decide si la aprueba o la modifica antes de su uso posterior. La pantalla incorpora indicadores de cobertura, clasificación de acciones y opciones como editar, copiar o guardar el script. En términos de gobernanza, esta etapa concreta la lógica human-in-the-loop: AURA no aplica una transformación de manera opaca, sino que exige un control humano explícito antes de continuar hacia la exportación o el uso del artefacto generado.

5.5. Fragmentos de código explicados

Los fragmentos siguientes no pretenden reemplazar la lectura completa del repositorio, sino mostrar de forma breve cómo se implementan los principios técnicos de AURA. Cada fragmento se acompaña de una explicación y de su límite.

5.5.1. Lectura local del CSV

El módulo de lectura local convierte el archivo en filas y columnas dentro del navegador. La función detecta campos y delimitador, y entrega el contenido al motor de auditoría.

```
const { data, meta } = await parseCsv(uploadedFile);  
// data: filas del CSV como objetos  
// meta.fields: nombres de columna  
// meta.delimiter: separador detectado
```

Este fragmento demuestra procesamiento local sin backend. No demuestra rendimiento con archivos de millones de filas.

5.5.2. Regla determinista representativa

Una regla determinista inspecciona valores de texto para detectar espacios invisibles al inicio o final. El hallazgo resultante incluye severidad, columna y conteo.

```
if (typeof value === 'string' && value !== value.trim()) {  
  issues.push({  
    ruleName: 'Espacios Fantasma (Trim)',  
    severity: 'info',  
    column: field,  
    count: 1,  
    affectedPercentage: ...  
  });  
}
```

Este fragmento demuestra reglas reproducibles. No demuestra que una sola regla cubra todos los casos posibles de limpieza textual.

5.5.3. Construcción del paquete de evidencia

Antes de la interpretación asistida, AURA construye un paquete con fingerprint, tiempos, columnas procesadas y resultado del informe técnico. Este paquete reduce la necesidad de exponer el dataset completo.

```
const datasetFingerprint = fingerprintDataset(data, meta.fields);
const evidence = buildAuditEvidence({
  fileName, datasetFingerprint,
  startedAt, completedAt,
  rowsProcessed, columnsProcessed,
  report: auditResult,
  trace: trace.events
});
```

Este fragmento demuestra trazabilidad del procesamiento. No demuestra resistencia absoluta frente a manipulación externa.

5.5.4. Generación de script desde acciones aprobadas

El generador controlado produce código únicamente para acciones aprobadas. Las acciones rechazadas o pendientes no generan transformaciones.

```
function renderActionV2(action, columnRef, columnRegistry): string {
  switch (action.actionType) {
    case 'trim_whitespace':
      return `df_clean[${writeTarget}] = df_clean[${writeTarget}].astype(str).str.strip()`;
    // otros tipos de acción
  }
}
```

Este fragmento demuestra que el código deriva de decisiones humanas y reglas internas, no de texto libre del LLM. No demuestra que el script generado sea óptimo para todos los dominios.

5.5.5. Revisión humana y bloqueo

Antes de aprobar el contrato de script, el sistema realiza una verificación fresca. Si hay inconsistencia, por ejemplo un hash alterado, la aprobación se bloquea.

```
const freshVerification = verifyScriptContractV2(
  scriptContractV2, remediationPlanV2, contextResult.buildContext
);
if (!freshVerification.valid) {
  setStage('pending');
  return; // bloquea la aprobación
}
```

Este fragmento demuestra una política de fallo cerrado ante inconsistencias. No demuestra que todas las formas posibles de manipulación sean detectadas.

6. Código fuente y datos analizados

6.1. Código fuente

El código fuente de AURA se organiza como un proyecto web de desarrollo software. La carpeta documental activa de la tercera entrega recoge tanto la memoria como la evidencia académica usada en este documento. Desde el punto de vista funcional, la aplicación ya expone un flujo de interfaz que incluye carga, perfilamiento, diagnóstico, preparación de script, revisión y exportación, además de un laboratorio de comparación de modelos. Esta evolución explica por qué la memoria combina descripción por fases con descripción del funcionamiento actual del prototipo.

6.2. Datos analizados

Esta entrega utiliza dos datasets principales. Titanic, con 891 filas y 12 columnas, se emplea como caso aplicado para demostrar carga local, perfilamiento, hallazgos, diagnóstico, plan de remediación y revisión humana. `synthetic_ground_truth.csv`, con 15 filas y 9 columnas, se emplea como dataset controlado con reglas conocidas para medir el motor determinista.

La diferencia entre ambos es importante. Titanic permite una demostración realista y reproducible, pero no contiene ground truth exhaustivo para todos los problemas de calidad. El dataset sintético sí permite medir verdaderos positivos, falsos positivos y falsos negativos, aunque su tamaño reducido limita la generalización a escenarios empresariales complejos.

7. Resultados consolidados de la tercera entrega

Los resultados de esta entrega se presentan en varios niveles complementarios. En primer lugar, se documenta el comportamiento del prototipo sobre Titanic para mostrar el flujo visible de uso: carga, perfilamiento, diagnóstico, propuesta de script, revisión humana y preparación para exportación. En segundo lugar, se utilizan resultados con ground truth para medir el desempeño del motor determinista en condiciones controladas. Finalmente, se resume el estado del laboratorio de modelos y de los proveedores actualmente soportados por la arquitectura.

7.1. Resultados sobre Titanic

Elemento evaluado	Resultado	Interpretación académica	Limitación
Carga y perfilamiento	891 filas y 12 columnas procesadas localmente	Demuestra que AURA puede leer un dataset público tabular sin backend.	No prueba escalabilidad con archivos masivos.

Hallazgos deterministas	10 hallazgos detectados	Incluye nulos, outliers, espacios fantasma y distribución de categorías.	Sin ground truth completo puede haber falsos positivos.
Paquete de evidencia	Fingerprint, columnas y hallazgos encapsulados	El diagnóstico asistido parte de evidencia acotada.	No garantiza por sí solo la calidad del modelo.
Diagnóstico asistido	Hipótesis vinculadas a hallazgos del motor	El modelo no trabaja sobre datos libres, sino sobre evidencia preparada.	La captura se apoya en un flujo determinista para reproducibilidad.
Plan de remediación	1 acción automática segura, 7 de revisión y 2 no accionables	La mayoría de acciones requieren criterio humano.	La clasificación depende del contexto de negocio.
Revisión humana	Acciones aprobadas y rechazadas individualmente	Evidencia separación entre propuesta y decisión.	No mide la calidad de la decisión humana.
Script propuesto	Código Python/Pandas generado desde acciones aprobadas	El script no nace de texto libre del LLM.	No fue ejecutado.
Validación del contrato	Partición aceptadas/rechazadas/excluidas y verificación previa	Permite confirmar coherencia antes de aprobar.	La sintaxis no fue validada por un intérprete Python.
Bloqueo por manipulación	Aprobación bloqueada ante inconsistencia	El sistema falla de forma cerrada.	Solo cubre el caso de manipulación probado.

Tabla 3. Resultados sobre Titanic.

El resultado central sobre Titanic es que AURA puede construir una cadena de evidencia sin afirmar todavía limpieza efectiva. El flujo muestra lectura local, hallazgos deterministas, diagnóstico estructurado, acciones propuestas, revisión humana y script revisable. Sin embargo, el script no fue ejecutado y no se realizó reauditoría posterior.

7.2. Resultados sobre dataset con ground truth

Métrica	Resultado	Criterio	Interpretación
Reglas evaluadas	16	Número de anomalías esperadas en el dataset sintético	El conjunto fue diseñado para validación controlada.
Hallazgos generados	16	Conteo de hallazgos emitidos por el motor determinista	Coincide con el volumen esperado de reglas.
Verdaderos positivos	≥ 15	Hallazgos que coinciden con errores conocidos	Alta capacidad de detección.
Falsos positivos	≤ 1	Alertas sin correspondencia directa con ground truth	Ruido bajo en el escenario controlado.
Falsos negativos	≤ 1	Errores conocidos no detectados	Pérdida baja en reglas evaluadas.

Precisión	$\geq 0,93$	$TP / (TP + FP)$	Cuando AURA reporta un hallazgo, casi siempre corresponde a un error esperado.
Recall	$\geq 0,93$	$TP / (TP + FN)$	AURA encuentra casi todos los errores inyectados.
F1-score	$\geq 0,90$	Media armónica entre precisión y recall	Resultado sólido para el motor determinista en dataset controlado.

Tabla 4. Resultados sobre dataset con ground truth.

La validación controlada permite corregir una afirmación importante de versiones anteriores: el motor determinista no debe describirse como de precisión perfecta. La formulación madura es que, bajo el dataset sintético controlado, el motor alcanza valores mínimos altos de precisión, recall y F1, pero su comportamiento debe seguir evaluándose por regla, umbral y dominio.

7.3. Estado del laboratorio de comparación de modelos

Modelo / proveedor	Estado	Latencia	Observación académica	Límite
DeepSeek cloud	Ejecutado en desarrollo	Variable	Respeto bien formato estructurado; usado como proveedor principal de pruebas.	No se declara ganador.
Gemini cloud	Ejecutado en desarrollo	Variable	Buen rendimiento general, con riesgo ocasional de campos extra.	Requiere corridas repetidas.
Ollama local	Prueba operativa	Depende del equipo	Útil para privacidad y operación sin conexión.	No demuestra calidad diagnóstica por sí solo.
WebLLM local	Experimental	Limitado por hardware	Puede mantener inferencia local.	No se recomienda aún como camino principal.
Chrome Prompt API	Experimental	Pocas pruebas	Prometedor como inferencia integrada al navegador.	Disponibilidad todavía limitada.

Tabla 5. Estado del laboratorio de comparación de modelos.

El laboratorio de modelos existe como infraestructura experimental y ya documenta compatibilidad operativa con proveedores cloud, locales y experimentales en navegador. Entre ellos destacan DeepSeek y Gemini como alternativas cloud, Ollama como opción local y Chrome Prompt API junto con WebLLM como opciones experimentales. Sin embargo, esta entrega no declara un benchmark formal ni un modelo ganador, porque todavía no se ha ejecutado un protocolo homogéneo con suficientes repeticiones y bajo las mismas condiciones para todos los proveedores.

7.4. Evidencia por objetivo específico

OE	Propósito	Evidencia	Resultado actual	Límite
OE1	Arquitectura local-first	Carga local, fingerprint y perfilamiento del CSV.	Implementado en el flujo de carga y perfilamiento.	No prueba escalabilidad con archivos de gran tamaño.
OE2	Motor determinista	Hallazgos sobre Titanic y validación sobre synthetic_ground_truth.	16 reglas evaluadas, precisión y recall $\geq 0,93$.	El dataset sintético es pequeño.
OE3	Diagnóstico asistido restringido	Paquete de evidencia y diagnóstico estructurado.	El modelo recibe evidencia limitada.	La evidencia visual usa flujo determinista, no inferencia LLM real completa.
OE4	Laboratorio LLM	Módulo de comparación de proveedores.	Laboratorio disponible y documentado.	No hay benchmark formal con repeticiones suficientes.
OE5	Gobernanza humana	Plan de remediación y revisión HITL.	El usuario aprueba o rechaza acciones.	No mide calidad de decisiones del usuario.
OE6	Script revisable	Script propuesto y contrato verificable.	Código generado desde acciones aprobadas.	Script no ejecutado.

Tabla 6. Relación entre objetivos, evidencia y límites.

8. Conclusiones

La tercera entrega consolida una evolución conceptual y técnica de AURA. La herramienta deja de plantearse como un sistema donde el LLM genera libremente diagnósticos y transformaciones, y se redefine como una arquitectura gobernada: el motor produce hechos, el paquete de evidencia limita el contexto, el diagnóstico interpreta, el plan propone, la persona decide y el generador controlado prepara código revisable.

Respecto al OE1, AURA mantiene un flujo local-first de carga y perfilamiento. Respecto al OE2, el motor determinista dispone de validación controlada sobre un dataset con ground truth, con precisión y recall superiores o iguales a 0,93 y F1 igual o superior a 0,90. Respecto al OE3, el diagnóstico asistido se restringe mediante evidencia estructurada. Respecto al OE4, el laboratorio de modelos está disponible, aunque sus resultados se mantienen como experimentales hasta ejecutar un protocolo formal. Respecto al OE5 y OE6, AURA incorpora revisión humana y preparación de scripts revisables, pero sin afirmar todavía ejecución ni mejora real del dataset.

La principal contribución de esta entrega no es prometer limpieza automática, sino demostrar una cadena auditable y honesta. AURA muestra cómo integrar reglas deterministas, modelos de lenguaje y revisión humana sin ceder control sobre los datos, las acciones ni las afirmaciones del sistema.

9. Limitaciones y prospectiva

9.1. Limitaciones

- Titanic no se considera ground truth completo; se usa como caso aplicado y reproducible.
- El dataset sintético permite medición controlada, pero su tamaño reducido limita la generalización.
- El diagnóstico asistido documentado en las capturas se apoya en evidencia estructurada reproducible, no en una evaluación completa de calidad LLM.
- El script propuesto no fue ejecutado y, por tanto, no se afirma mejora efectiva del dataset.
- No se realizó reauditoría posterior ni comparación antes/después.
- El laboratorio de modelos aún no cumple las condiciones de benchmark formal.

9.2. Trabajo futuro

La fase final del desarrollo se orientará a cerrar el ciclo completo de AURA. Esto implica ejecutar de forma controlada los scripts aprobados sobre una copia del dataset, comparar el estado antes y después de la limpieza, y documentar con evidencia si las acciones propuestas mejoran efectivamente la calidad del dato. En paralelo, el laboratorio de comparación de modelos deberá afinarse como módulo estable de experimentación, con reglas homogéneas de evaluación, repetición suficiente y criterios claros para interpretar resultados sin sobreprometer capacidades.

También se debe cerrar el laboratorio de comparación de modelos con un protocolo homogéneo: mismo dataset, mismo paquete de evidencia, temperatura controlada, número suficiente de repeticiones y métricas comparables. Solo después de ese protocolo será válido declarar diferencias entre proveedores o modelos.

Finalmente, se recomienda realizar una evaluación de usabilidad con usuarios objetivo. Esa evaluación debería medir tiempo por tarea, comprensión de hallazgos, confianza en el script propuesto y claridad de los límites del sistema.

Referencias bibliográficas

- Akbasli, I. T., Birbilen, A. Z., & Teksam, O. (2025). Leveraging large language models to mimic domain expert labeling in unstructured text-based electronic healthcare records in non-English languages. *BMC Medical Informatics and Decision Making*, 25, 1–9. <https://doi.org/10.1186/s12911-025-02871-6>
- Aldoseri, A., Al-Khalifa, K. N., & Hamouda, A. M. (2023). Re-thinking data quality for AI: A timeliness dimension. *Computers in Industry*, 173. <https://doi.org/10.1016/j.compind.2023.103994>
- Banitalebi, B., et al. (2025). A framework for trust in AI-driven data observability and quality control systems. *IEEE Conference on Artificial Intelligence (CAI)*. <https://doi.org/10.1109/CAI62590.2025.11050490>
- Davenport, T. (2025). A taxonomy of AI-augmented frameworks for data quality validation. *International Journal of Applied AI Research*. <https://aimjournals.com/index.php/ijaair/article/view/382>
- España. (2018). Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. *Boletín Oficial del Estado*. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- European Parliament and Council of the European Union. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence. *Official Journal of the European Union*. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- Gartner, Inc. (2024). Gartner predicts 30% of generative AI projects will be abandoned after proof of concept by end of 2025. <https://www.gartner.com/en/newsroom/press-releases/2024-07-29-gartner-predicts-30-percent-of-generative-ai-projects-will-be-abandoned-after-proof-of-concept-by-end-of-2025>
- Hilmi, A., et al. (2026). Hybrid rule-based and on-premises LLM pipeline for extracting CMR and CPET metrics from free-text reports. *medRxiv*. <https://doi.org/10.1101/2026.01.21.26344531>
- IBM Institute for Business Value. (2026). The true cost of poor data quality. <https://www.ibm.com/think/insights/cost-of-poor-data-quality>
- ISO/IEC. (2008). ISO/IEC 25012:2008 Software engineering: Data quality model. <https://www.iso.org/standard/35736.html>

- Jakubik, J., Vössing, M., Kühn, N., Walk, J., & Satzger, G. (2024). Data-centric artificial intelligence. *Business & Information Systems Engineering*. <https://doi.org/10.1007/s12599-024-00857-8>
- Li, L., Fang, L., & Torvik, V. I. (2025). AutoDCWorkflow: LLM-based data cleaning workflow auto-generation and benchmark. *Findings of EMNLP 2025*. <https://arxiv.org/abs/2412.06724>
- Li, X., Cheng, Y., Lee, J., & Møller, C. (2025). Data issues in industrial AI systems: A systematic literature review. *Computers in Industry*, 173. <https://doi.org/10.1016/j.compind.2025.104201>
- Moses, B. (2025). State of data quality survey: The rise of data downtime. *Monte Carlo Data*. <https://www.montecarlodata.com/blog-data-quality-survey>
- Ng, A. (2024). Data-centric artificial intelligence (v4). *arXiv*. <https://arxiv.org/abs/2212.11854>
- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo. (2016). Reglamento General de Protección de Datos. *Diario Oficial de la Unión Europea*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Ruan, C. F., Qin, Y., Zhou, X., Lai, R., Jin, H., Dong, Y., Hou, B., Yu, M.-S., Zhai, Y., Agarwal, S., Cao, H., Feng, S., & Chen, T. (2024). WebLLM: A high-performance in-browser LLM inference engine. *arXiv*. <https://doi.org/10.48550/arXiv.2412.15803>
- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P. K., & Aroyo, L. (2021). Data cascades in high-stakes AI. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3411764.3445518>
- Whang, S. E., Roh, Y., & Lee, J. G. (2023). Data collection and quality challenges in deep learning: A data-centric AI perspective. *VLDB Journal*, 32(4), 791–813. <https://doi.org/10.1007/s00778-022-00775-9>
- Zha, D., Bhat, Z. P., Lai, K. H., Yang, F., & Hu, X. (2025). Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5), Article 129. <https://doi.org/10.1145/3711118>
- Zhang, S., Huang, Z., & Wu, E. (2024). Data cleaning using large language models. *ICDEW 2025*. <https://arxiv.org/abs/2410.15547>
- Zhou, X., et al. (2026). Can LLMs clean up your mess? A survey of application-ready data preparation with LLMs. *arXiv*. <https://arxiv.org/abs/2601.17058>

Anexo A. Privacidad y protección de datos

Antes de procesar un dataset real con AURA, el usuario debe identificar la finalidad del tratamiento, el tipo de datos incluidos, la existencia de categorías especiales, el modo de inferencia seleccionado y las medidas de minimización aplicables. Si el procesamiento se mantiene local, el riesgo de exposición externa se reduce. Si se utiliza un proveedor cloud, debe verificarse si el paquete de evidencia contiene datos personales, muestras sensibles o metadatos identificables.

Anexo B. Síntesis de evidencia consolidada

Evidencia	Estado	Qué demuestra	Límite principal
Capturas de perfilamiento y diagnóstico	Incorporadas	Flujo visual del prototipo y separación de etapas	No demuestran mejora del dataset.
Resultados sobre Titanic	Consolidados para esta entrega	Aplicabilidad sobre dataset público de 891 filas	No existe ground truth exhaustivo.
Resultados synthetic_ground_truth	Validación controlada	Métricas del motor determinista con reglas conocidas	Dataset pequeño y sintético.
Fragmentos de código	Incorporados	Trazabilidad entre diseño y software	No sustituyen revisión completa del repositorio.
Laboratorio de modelos	Experimental	Infraestructura de comparación disponible	No hay benchmark formal ni modelo ganador.

Tabla 7. Síntesis de evidencia consolidada para la tercera entrega.

Anexo C. Claims permitidos y no permitidos

Claims permitidos: AURA perfila datasets localmente; genera hallazgos deterministas reproducibles; estructura evidencia antes del diagnóstico asistido; separa diagnóstico, propuesta y decisión humana; genera scripts revisables desde acciones aprobadas; exige revisión humana antes de ejecutar acciones; mantiene el laboratorio de modelos como experimental.

Claims no permitidos: AURA corrige definitivamente el dataset; AURA ejecuta Python en producción; AURA calcula una mejora real sin reauditoría; AURA elimina alucinaciones; existe benchmark formal completado; un modelo es superior a otro sin protocolo de repetición.

Anexo D. Explicación extendida de las etapas de AURA

Este anexo complementa el Capítulo 5 con una explicación más detallada de las etapas internas de AURA. Su propósito es evitar que la memoria dependa de nombres de archivos o clases de software sin contexto. Cada etapa se explica desde la perspectiva funcional: qué problema resuelve, qué entrada recibe, qué salida produce y qué riesgo controla.

D.1. Etapa de carga local

La carga local es el punto de entrada del sistema. El usuario selecciona un archivo CSV y AURA lo interpreta dentro del navegador. Esta decisión arquitectónica reduce la exposición del archivo crudo, evita la dependencia inicial de un backend y permite que el primer diagnóstico se produzca sin enviar los datos a un proveedor externo. En términos académicos, esta etapa responde a la preocupación por privacidad, minimización y control operativo del tratamiento.

La salida de esta etapa no es todavía un diagnóstico. La herramienta solo conoce filas, columnas, delimitador y estructura inicial. Este límite es importante: leer un archivo correctamente no implica que el dataset sea de calidad ni que sus errores estén resueltos. Por ello, la carga local se conecta inmediatamente con el motor determinista de auditoría.

D.2. Etapa de perfilamiento determinista

El perfilamiento determinista constituye la primera lectura técnica del dataset. AURA aplica reglas explícitas, expresiones regulares, heurísticas de tipos y estadística descriptiva para detectar problemas observables. Entre los hallazgos posibles aparecen valores nulos, duplicados, espacios invisibles, categorías poco frecuentes, valores extremos y formatos no esperados.

La ventaja de este enfoque es la reproducibilidad: si se entrega el mismo dataset bajo la misma versión de reglas, el resultado esperado debe ser el mismo. La limitación es que una regla puede generar falsos positivos cuando el dominio de negocio justifica un valor aparentemente raro. Por ello, el motor determinista no se presenta como juez final, sino como generador de evidencia objetiva para las siguientes etapas.

D.3. Etapa de empaquetado de evidencia

El paquete estructurado de evidencia es el puente entre el análisis determinista y el diagnóstico asistido. En lugar de entregar el dataset completo al modelo de lenguaje, AURA prepara una

representación reducida: columnas, hallazgos, conteos, severidades, muestras limitadas y metadatos de ejecución. Esta decisión reduce exposición de datos y limita el espacio en el que el modelo puede formular afirmaciones.

Desde el punto de vista metodológico, este paquete es una barrera contra la improvisación del modelo. El diagnóstico asistido debe apoyarse en los hallazgos existentes y no en conocimiento externo o suposiciones. Si una columna no existe o una evidencia no fue generada por el motor, no debería aparecer como fundamento de una conclusión.

D.4. Etapa de diagnóstico asistido

El diagnóstico asistido cumple una función interpretativa. Su tarea no es volver a perfilar el dataset ni decidir transformaciones, sino ayudar a explicar qué podrían significar los hallazgos. Por ejemplo, un campo con muchos nulos puede indicar ausencia real de información, error de captura, campo opcional o problema de integración. El valor del modelo está en formular hipótesis útiles, no en reemplazar la evidencia.

Esta etapa también debe ser leída con cautela. Un modelo de lenguaje puede redactar explicaciones plausibles aunque no estén completamente soportadas. Por ello, AURA conserva límites explícitos, referencias a hallazgos y revisión posterior. La interpretación asistida mejora comprensión, pero no autoriza cambios por sí sola.

D.5. Etapa de plan de remediación

El plan de remediación traduce hallazgos e interpretaciones en acciones posibles. Aquí AURA adopta una política conservadora: una acción puede ser automática segura, requerir revisión o quedar como no accionable. Esta clasificación evita que una recomendación textual se convierta directamente en una transformación.

La decisión más importante de esta etapa es separar diagnóstico y acción. Aunque el modelo ayude a interpretar un problema, la selección de una acción se apoya en reglas internas y control humano. Esto reduce el riesgo de que un modelo sugiera borrar, reemplazar o imputar valores sin justificación suficiente.

D.6. Etapa de revisión humana

La revisión humana introduce responsabilidad profesional en el flujo. El usuario puede aprobar o rechazar acciones antes de que se genere código. Esta revisión no debe verse como un simple botón de confirmación, sino como una salvaguarda metodológica: el sistema muestra qué pretende hacer, sobre qué columna, con qué tipo de acción y bajo qué nivel de riesgo.

La revisión humana tampoco garantiza automáticamente una decisión correcta. Sin embargo, mejora la gobernanza porque impide que el modelo o el sistema actúen sin autorización. En futuros trabajos, esta etapa debería evaluarse con usuarios reales para medir claridad, confianza y comprensión de límites.

D.7. Etapa de script revisable

El script revisable representa la preparación técnica de la limpieza. AURA genera código Python/Pandas a partir de acciones aprobadas, no desde texto libre del modelo. El objetivo es que el código sea trazable: cada línea relevante debe corresponder a una acción previamente aceptada.

En esta tercera entrega, el script se presenta como propuesta revisable y no como limpieza ejecutada. Esta distinción es esencial. Un script generado y aprobado no equivale a un dataset mejorado; solo indica que existe una propuesta técnica lista para una etapa posterior de ejecución controlada.

D.8. Etapa final pendiente: ejecución controlada y comparación antes/después

La fase final del proyecto deberá ejecutar el script aprobado sobre una copia del dataset original, nunca sobre el archivo fuente sin respaldo. Después de la ejecución, AURA deberá repetir la auditoría y comparar el estado antes y después. Solo con esta comparación podrán formularse conclusiones sobre mejora efectiva del dataset.

El resultado esperado de esa fase no debe limitarse a un porcentaje global. Será necesario mostrar qué reglas mejoraron, cuáles se mantuvieron igual, cuáles empeoraron y qué transformaciones fueron bloqueadas. Esa evidencia permitirá cerrar el ciclo completo: detección, explicación, propuesta, aprobación, ejecución y evaluación.

Anexo E. Protocolo de evaluación final propuesto

El cierre final del TFM requiere un protocolo que evite mezclar evidencias de naturaleza distinta. La evaluación del motor determinista, la evaluación del diagnóstico asistido, la evaluación del script y la evaluación del laboratorio de modelos deben separarse. Cada una responde a una pregunta diferente y requiere métricas propias.

Dimensión	Pregunta	Entrada	Métrica principal	Límite
Motor determinista	¿Detecta correctamente anomalías conocidas?	Dataset con ground truth	TP, FP, FN, precisión, recall, F1	No mide calidad LLM.
Diagnóstico asistido	¿Explica hallazgos sin inventar evidencia?	Paquete estructurado de evidencia	Referencias válidas, claims sin soporte, cumplimiento de formato	No autoriza transformaciones.
Plan de remediación	¿Clasifica acciones de forma gobernada?	Hallazgos y diagnóstico estructurado	Acciones auto-safe, review-only, not-actionable	No ejecuta limpieza.
Script revisable	¿Genera código trazable desde acciones aprobadas?	Acciones aprobadas	Cobertura, partición, verificación previa	No prueba mejora si no se ejecuta.
Ejecución controlada	¿Mejora realmente el dataset?	Copia del dataset original	Comparación antes/después, reauditoría, variación de salud	Pendiente en esta entrega.
Laboratorio de modelos	¿Qué modelo responde mejor bajo protocolo común?	Mismo dataset, mismo paquete, repeticiones	Latencia, formato, alucinaciones, utilidad diagnóstica	No declarar ganador sin protocolo formal.

Este protocolo permitirá que la fase final no se limite a mostrar que AURA funciona visualmente. La meta será demostrar qué parte del sistema detecta, qué parte interpreta, qué parte propone, qué parte ejecuta y qué parte mejora efectivamente el dato.